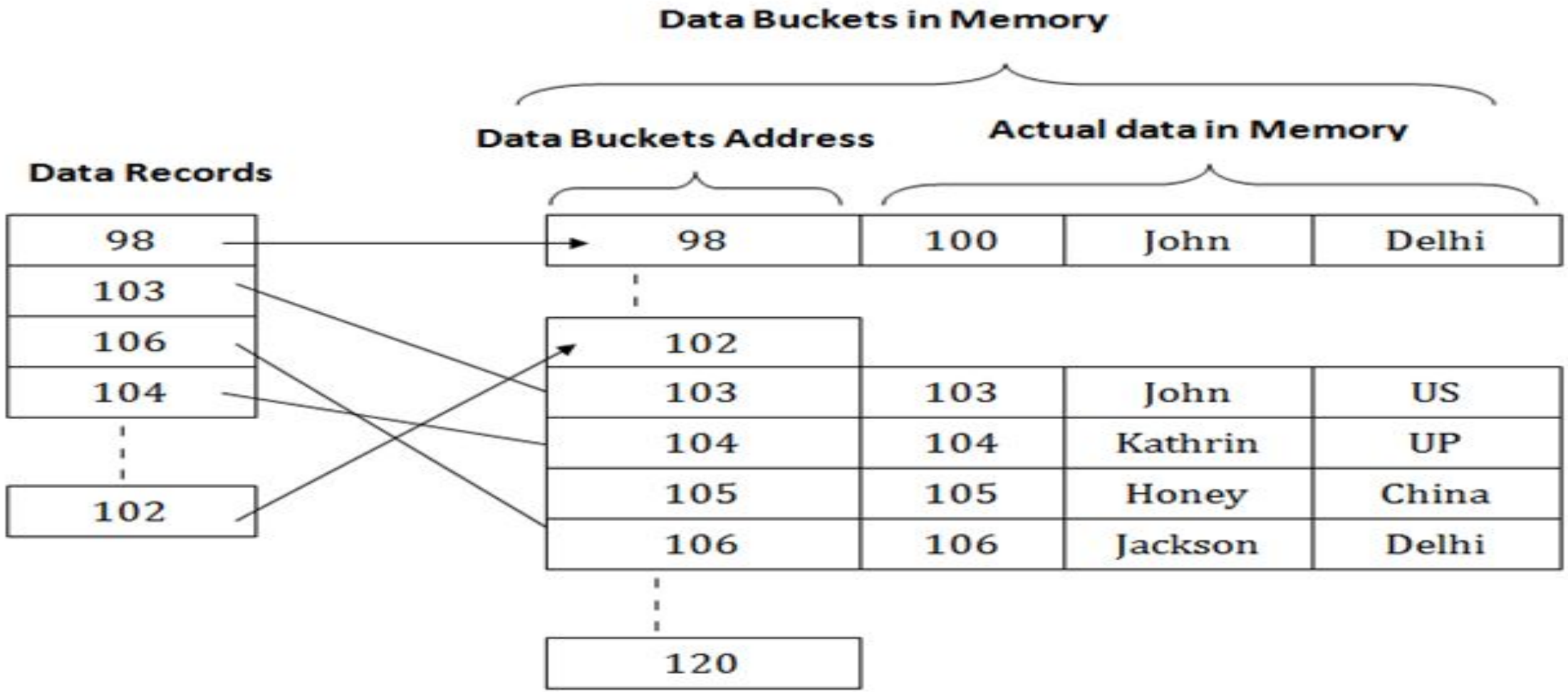
Hashing

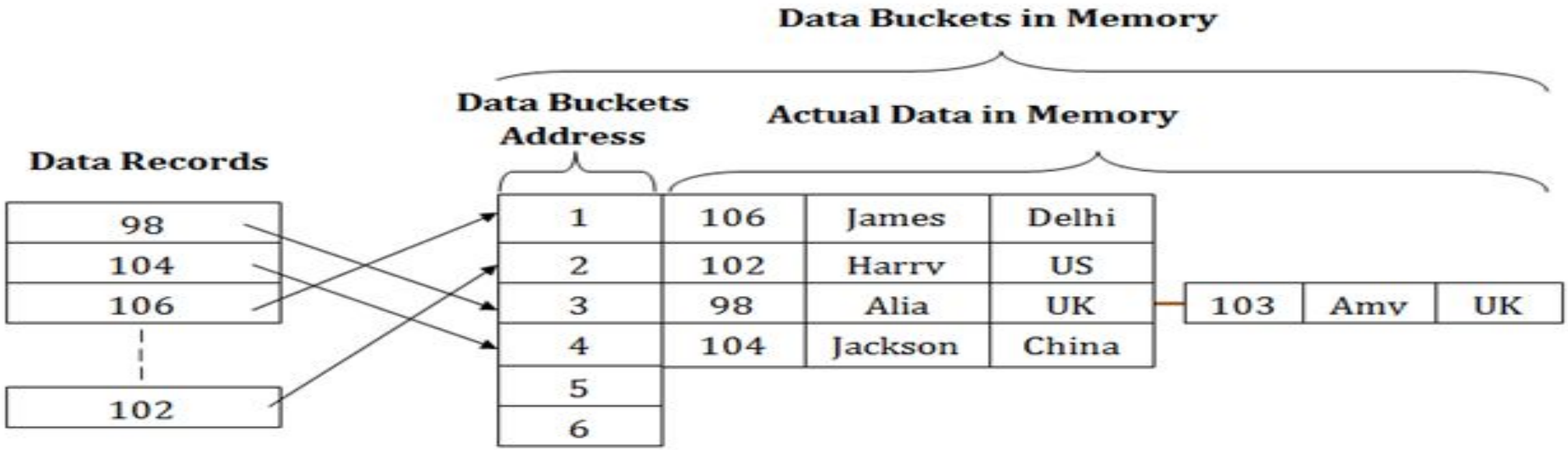
- In a huge database structure, it is very inefficient to search all the index values and reach the desired data.
- Hashing technique is used to calculate the direct location of a data record on the disk without using index structure.
- In this technique, data is stored at the data blocks whose address is generated by using the hashing function. The memory location where these records are stored is known as data bucket or data blocks.
- The hash function uses the primary key to generate the address of the data block
- A hash function is a simple mathematical function to any complex mathematical function. We can even consider the primary key itself as the address of the data block. That means each row whose address will be the same as a primary key stored in the data block.

Hashing

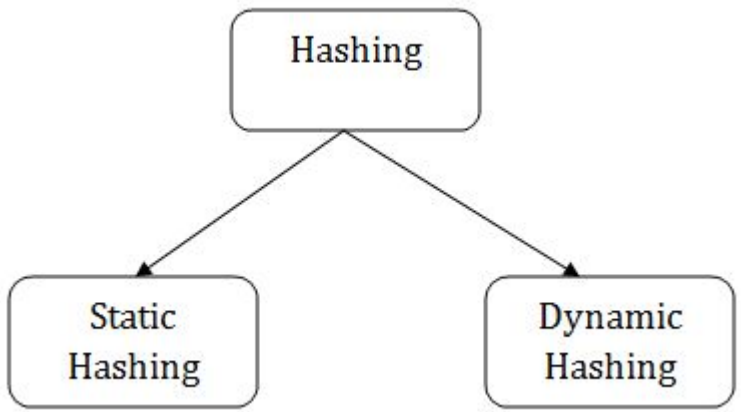

Data Buckets in Memory
Data Buckets Address
Actual data in Memory
Data Records
98
103
106
104
102
98
100
John
Delhi
102
103
103
John
US
104
104
Kathrin
UP
105
105
Honey
China
106
106
Jackson
Delhi
120

Hashing

- The above diagram shows data block addresses same as primary key value.
- This hash function can also be a simple mathematical function like exponential, mod, cos, sin, etc.
- Suppose we have mod (5) hash function to determine the address of the data block. In this case, it applies mod (5) hash function on the primary keys and generates 3, 3, 1, 4 and 2 respectively, and records are stored in those data block addresses.

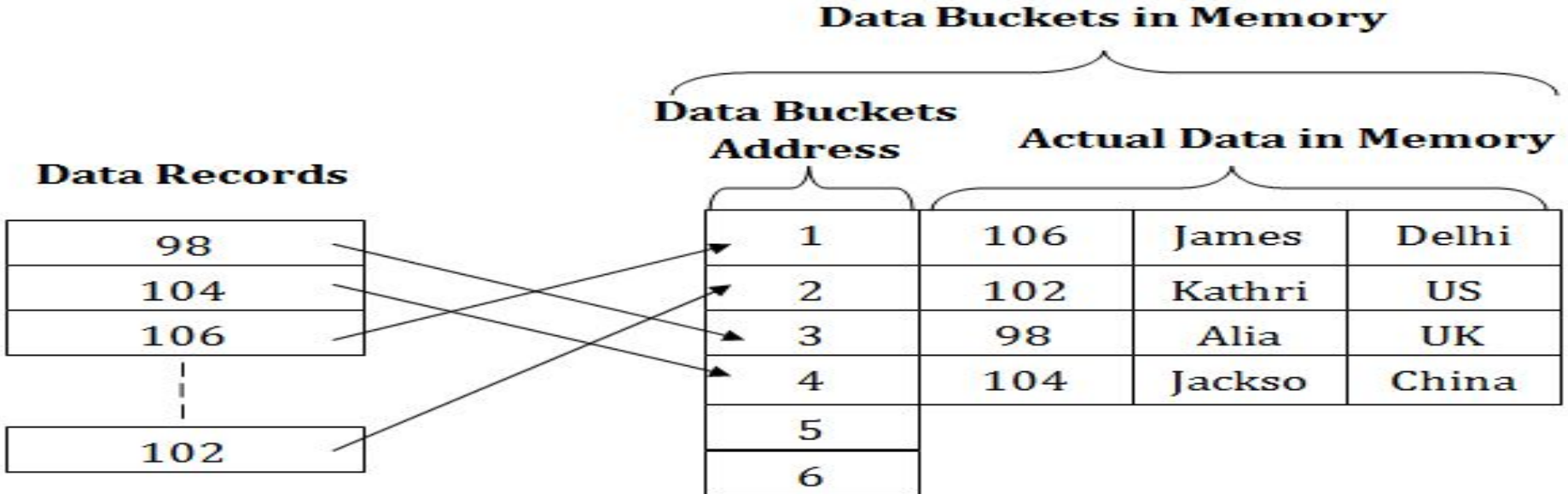


Types of Hashing:



Static Hashing:

- In static hashing, the resultant data bucket address will always be the same
- If we generate an address for EMP_ID =103 using the hash function mod (5) then it will always result in same bucket address 3. Here, there will be no change in the bucket address.
- In static hashing, the number of data buckets in memory remains constant throughout
- In this example, we will have five data buckets in the memory used to store the data.



operations on Static Hashing

- **Searching a record**

  When a record needs to be searched, then the same hash function retrieves the address of the bucket where the data is stored

- **Insert a Record**

  When a new record is inserted into the table, then we will generate an address for a new record based on the hash key and record is stored in that location

- **Delete a Record**

  To delete a record, we will first fetch the record which is supposed to be deleted. Then we will delete the records for that address in memory

- **Update a Record**

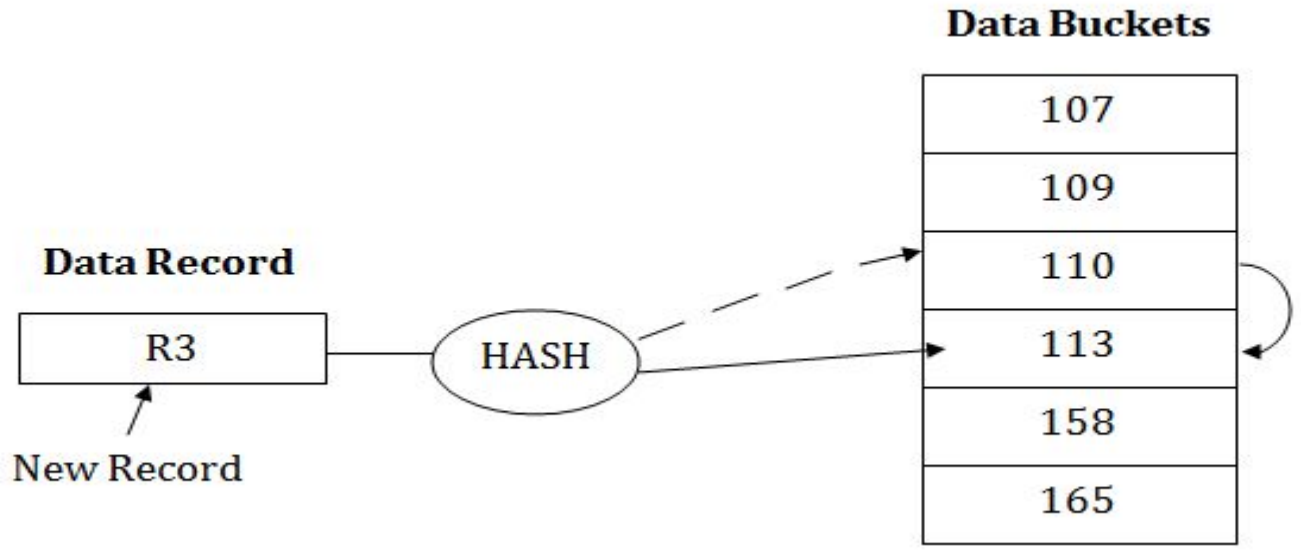
  To update a record, we will first search it using a hash function, and then the data record is updated

operations on Static Hashing

- If we want to insert some new record into the file but the address of a data bucket generated by the hash function is not empty, or data already exists in that address. This situation in the static hashing is known as **bucket overflow**. This is a critical situation in this method.

1. Open Hashing

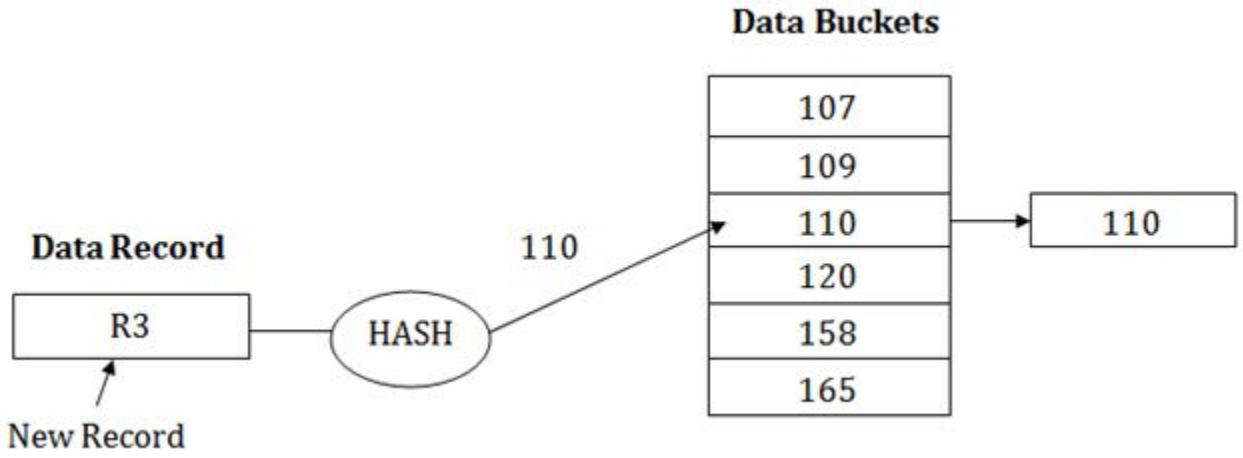
- When a hash function generates an address at which data is already stored, then the next bucket will be allocated to it. This mechanism is called as **Linear Probing**.
- **For example:** suppose R3 is a new address which needs to be inserted, the hash function generates address as 110 for R3. But the generated address is already full. So the system searches next available data bucket, 113 and assigns R3 to it.



operations on Static Hashing

2. Close Hashing

- When buckets are full, then a new data bucket is allocated for the same hash result and is linked after the previous one. This mechanism is known as **Overflow chaining**.
- **For example:** Suppose R3 is a new address which needs to be inserted into the table, the hash function generates address as 110 for it. But this bucket is full to store the new data. 0
- In this case, a new bucket is inserted at the end of 110 buckets and is linked to it.



Dynamic Hashing

- The dynamic hashing method is used to overcome the problems of static hashing like bucket overflow.

- In this method, data buckets grow or shrink as the records increases or decreases. This method is also known **as Extendable hashing method**.

- This method makes hashing dynamic, i.e., it allows insertion or deletion without resulting in poor performance.

How to search a key

- First, calculate the hash address of the key.

- Check how many bits are used in the directory, and these bits are called as i.

- Take the least significant i bits of the hash address. This gives an index of the directory.

- Now using the index, go to the directory and find bucket address where the record might be.

Dynamic Hashing

How to insert a new record

- First, calculate the hash address of the key
- If there is still space in that bucket, then place the record in it
- If the bucket is full, then we will split the bucket and redistribute the records

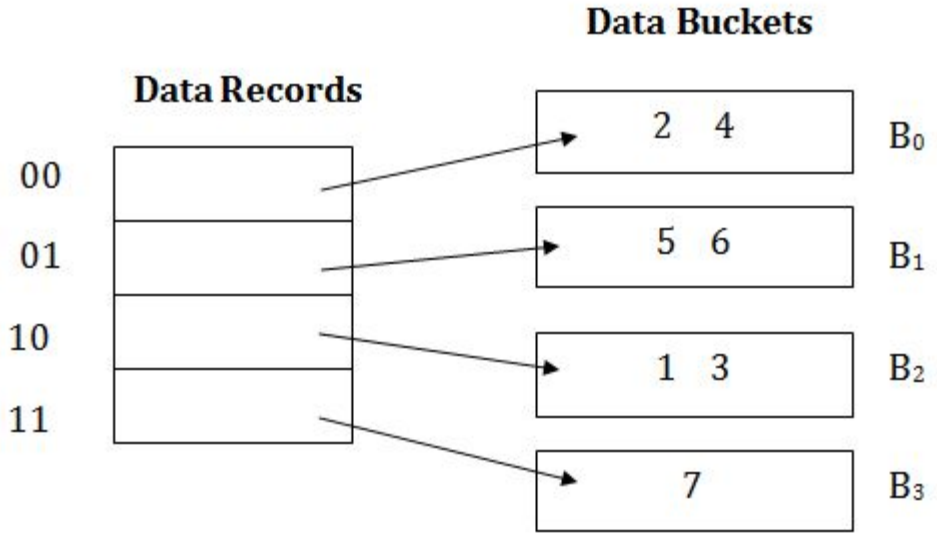
For example:
Consider the following grouping of keys into buckets, depending on the prefix of their hash address:

Key	Hash address
1	11010
2	00000
3	11110
4	00000
5	01001
6	10101
7	10111

Dynamic Hashing

How to insert a new record

The last two bits of 2 and 4 are 00. So it will go into bucket B0. The last two bits of 5 and 6 are 01, so it will go into bucket B1. The last two bits of 1 and 3 are 10, so it will go into bucket B2. The last two bits of 7 are 11, so it will go into B3.

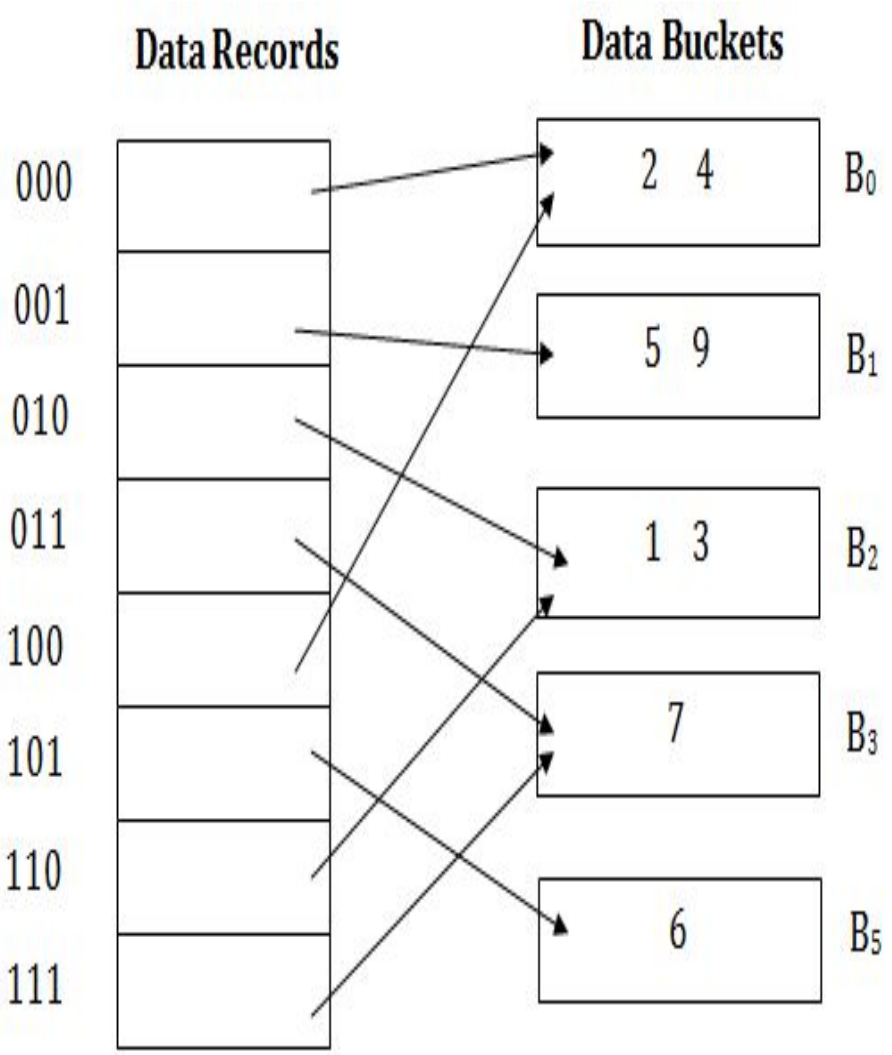


Dynamic Hashing

How to insert a new record

Insert key 9 with hash address 10001 into the above structure:

- Since key 9 has hash address 10001, it must go B1. But bucket B1 is full, so it will get split.
- The splitting will separate 5, 9 from 6 since last three bits of 5, 9 are 001, so it will go into bucket B1, and the last three bits of 6 are 101, so it will go into bucket B5.
- Keys 2 and 4 are still in B0. The record in B0 pointed by the 000 and 100 entry because last two bits of both the entry are 00.
- Keys 1 and 3 are still in B2. The record in B2 pointed by the 010 and 110 entry because last two bits of both the entry are 10.
- Key 7 are still in B3. The record in B3 pointed by the 111 and 011 entry because last two bits of both the entry are 11.



Dynamic Hashing

Advantages of dynamic hashing

- In this method, the performance does not decrease as the data grows in the system. It simply increases the size of memory to accommodate the data.

- In this method, memory is well utilized as it grows and shrinks with the data. There will not be any unused memory lying.

- This method is good for the dynamic database where data grows and shrinks frequently.

Disadvantages of dynamic hashing

- In this method, if the data size increases then the bucket size is also increased. These addresses of data will be maintained in the bucket address table. This is because the data address will keep changing as buckets grow and shrink. If there is a huge increase in data, maintaining the bucket address table becomes tedious.

- In this case, the bucket overflow situation will also occur. But it might take little time to reach this situation than static hashing.